



AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Project Proposal

O N

**Large-Scale Amazon Customer Review
Analytics using PySpark**

PREPARED FOR

Mr. Md Hasan Al Kayem

Lecturer Grade-I

Department of CSE, AUST

Dr. Afsana Ahmed Munia

Assistant Professor

Department of CSE, AUST

Course No: **CSE 4262**

Course Name: **Data Analytics Lab**

PREPARED BY

Lab Group: Gr-03

20220104006

A.S.M. Tahsin Tajware

20220104014

Abdullah Al Tamim

20220104032

Eusha Ahmed Mahi

Date: July 21, 2026

Abstract

A marketplace the size of Amazon produces millions of product reviews, turning the review corpus itself into a dataset worth mining rather than just a page shoppers scroll past. Once a corpus reaches that scale, single-machine tools such as Pandas begin to strain, exactly the gap distributed engines like Apache Spark are designed to close by keeping large, repeatedly-reused computations resident in memory across a cluster [8, 9]. This project proposes a PySpark-based analytics pipeline built on a working extract of the Amazon Reviews 2023 dataset released by the McAuley Lab at UC San Diego [1]: 521,607 reviews across the Cell Phones, Video Games, and Beauty categories, spanning September 1999 to September 2023. Profiling this extract confirms that verified purchases make up 93.1% of records, five-star ratings account for 61.3% of the corpus, a skew consistent with the long-observed positivity bias in online reviews [3], and that five-star reviews run noticeably shorter than lower-rated ones. Building on this foundation, the proposed pipeline uses Spark SQL and Spark’s window-function API to examine customer satisfaction, verified-purchase behaviour, review helpfulness, product- and reviewer-level rankings, temporal trends, and review-text characteristics, and closes with a timed comparison between PySpark and Pandas so the case for distributed processing rests on measured evidence rather than assumption.

Keywords: customer review analytics, PySpark, Apache Spark, distributed data processing, e-commerce, verified purchase, review helpfulness, window functions, performance benchmarking

Table of Contents

1	Introduction and Problem Statement	3
1.1	Background and Motivation	3
1.2	Problem Statement	3
2	Objectives	3
3	Dataset Description	4
3.1	Source and Summary	4
3.2	Category and Rating Distribution	5
3.3	Engagement, Length, and Schema	6
3.4	Scope and Limitations	6
4	Proposed Methodology	6
4.1	Data Preprocessing	6
4.2	Feature Engineering	7
4.3	Planned Analytics Modules	7
4.4	Performance Benchmarking (Objective 9)	7
4.5	System Workflow	7
5	Evaluation Strategy	8
6	Expected Outcomes and Deliverables	8
7	Conclusion	9
	References	9

1 Introduction and Problem Statement

1.1 Background and Motivation

Before adding an item to their cart, shoppers on a platform like Amazon typically read what previous buyers said about it. Chevalier and Mayzlin’s early analysis of Amazon and Barnes and Noble book sales found that review volume and valence moved together with sales, and that negative reviews carried more weight than positive ones of equal size [3]. The 521,607-review extract this project works with, drawn from the Amazon Reviews 2023 dataset [1] across Cell Phones, Video Games, and Beauty (1999–2023), reproduces the same positivity skew: 61.3% of reviews carry the maximum five-star rating (Section 4.2), treated here as a normal feature of real review data rather than a defect.

That same scale is what makes the corpus hard to work with using ordinary tools: a Pandas script comfortable with a few thousand rows becomes slow and memory-constrained once a dataset reaches hundreds of thousands. Apache Spark and its Python API, PySpark, close this gap through a reusable, in-memory working set [8], later formalized as the Resilient Distributed Dataset (RDD) [9]. Prior work confirms this is practical at Amazon-review scale: Khatun et al. process roughly 3.5 million Amazon reviews through a PySpark pipeline for spam detection [6], and Gupta and Kumari’s benchmarking of Spark against single-machine tooling supports the same conclusion more generally [7].

A further complication is that not every review can be taken at face value. Amazon’s “Verified Purchase” badge signals an actual purchase, and 93.1% of reviews in this extract carry it (Section 4.1). He, Hollenbeck, and Proserpio’s study of an organized market for fake reviews found that many manipulated reviews still carry this badge, since sellers can supply a free unit in exchange for a positive review without violating Amazon’s rule [4]. This project therefore treats verified-purchase status as something to examine empirically rather than as a settled trust signal.

1.2 Problem Statement

A team responsible for a marketplace this size needs concrete, quantitative answers to a cluster of questions rather than a single predictive score: How satisfied are customers overall, and does that hold steady across categories as different as Cell Phones, Video Games, and Beauty? Do verified and non-verified purchasers behave differently? Which reviews earn the most engagement? How have review volume and rating moved over the dataset’s 24-year span? Which specific products and reviewers stand out? And what does the review text reveal when measured directly?

Two gaps make these harder to answer than they first appear. First, the 521,607-record scale (Section 4) pushes single-node tools like Pandas toward their limits, particularly once several questions must be answered jointly. Second, most published work on review corpora targets a single predictive task, typically helpfulness prediction [5] or spam detection [4, 6], rather than the fuller set of descriptive questions a product team actually works through. This project addresses both gaps: it implements the analytics above in PySpark across a coherent set of business-relevant questions, and closes with a timed benchmark against Pandas so that PySpark’s advantage at this scale is demonstrated with measured evidence rather than taken on faith.

2 Objectives

The overarching goal of this project is to build and validate a PySpark-based analytics pipeline that turns a large, real-world corpus of e-commerce reviews into a coherent set of business-relevant findings, while giving the group hands-on practice with the distributed-processing techniques this course covers. That goal breaks down into the following objectives.

1. **Customer satisfaction analysis.** Quantify the marketplace-wide average rating and full 1–5 star distribution, and test whether it holds steady across the Cell Phones, Video Games, and Beauty categories.
2. **Verified-purchase analysis.** Compare verified and non-verified reviews on average rating, helpful-vote count, and review length, addressing the trust concerns raised in Section 2.
3. **Helpful-review analysis.** Surface the reviews with the highest helpful-vote counts, and compute average helpful-vote count overall and by star rating.
4. **Temporal trend analysis.** Track how review volume and average rating have moved year over year across the dataset’s 1999–2023 span, with attention to the sharp rise around 2019–2021.
5. **Product-level analytics.** Rank products by average rating and review count, applying a minimum-review-count threshold so low-volume products cannot rank top by chance.
6. **Customer-activity analysis.** Identify the most active reviewers and how review activity is distributed across the extract’s 500,000+ unique reviewers.
7. **Text analytics.** Measure review length against star rating, extract the most frequent corpus terms, and, as a lightweight extension, apply lexicon-based sentiment scoring against actual star rating.
8. **Window-function-based ranking.** Apply Spark’s window-function API to rank products within category, rank reviewers by activity, and compute a rolling average rating per product over time.
9. **PySpark-versus-Pandas benchmarking.** Time core operations, grouping, filtering, sorting, aggregation, in both frameworks to quantify rather than assume PySpark’s performance benefit at this scale.

Objectives 1–7 are the descriptive questions this project answers about the dataset itself; Objective 8 demonstrates a more advanced class of PySpark technique beyond a simple groupBy; and Objective 9 supplies the empirical justification for using PySpark in the first place. Section 5 maps each PySpark concept used to the objective it supports.

3 Dataset Description

3.1 Source and Summary

This project uses a real-world, three-category extract (Cell Phones, Video Games, Beauty) of the Amazon Reviews 2023 dataset, curated by the McAuley Lab at UC San Diego [1] and distributed on Hugging Face [2]. Its size requires distributed processing rather than a single-machine script. Table 1 profiles the extract as loaded, before cleaning – important for a groupBy-heavy pipeline, since an unhandled null key can silently vanish from an aggregation.

Table 1: Dataset summary

Attribute	Value
Total records	521,607 customer reviews
Date range	Sept 1999 – Sept 2023 (24 years)
Categories	Cell Phones, Video Games, Beauty
Verified / non-verified	485,784 (93.1%) / 35,823 (6.9%)
Unique reviewers / products	504,856 / 227,791
Missing values	186 review text, 156 review title (< 0.04%)

3.2 Category and Rating Distribution

The categories are unevenly represented (Table 2), with Cell Phones alone accounting for close to four out of five reviews – a scope note, not a defect (Section 4.4).

Table 2: Category distribution

Category	Reviews	Share
Cell Phones	415,467	79.7%
Video Games	91,966	17.6%
Beauty	14,174	2.7%
Total	521,607	100.0%

Ratings are J-shaped, not bell-shaped (Table 3): 5-star reviews dominate (61.3%), 1-star is next largest (13.3%), and middle ratings are rare – matching the positivity skew Chevalier and Mayzlin found in online book reviews [3]. Because average rating alone compresses most products into a narrow, high band, product ranking (Objective 5) will need a minimum-review-count threshold. Table 4 shows Video Games has both the lowest verified-purchase share and the highest average rating – worth revisiting after Objective 2.

Table 3: Rating distribution (marketplace-wide)

Rating	Count	Share
1 star	69,518	13.3%
2 star	30,771	5.9%
3 star	38,826	7.4%
4 star	62,515	12.0%
5 star	319,977	61.3%
Total	521,607	100.0%

Table 4: Average rating by category

Category	Avg. Rating	Verified Share
Video Games	4.05	86.0%
Cell Phones	4.02	94.8%
Beauty	3.96	90.5%
Marketplace-wide	4.02	93.1%

3.3 Engagement, Length, and Schema

Helpful-vote counts are heavily skewed toward zero: median and 75th percentile are both 0, only 17.1% of reviews get any helpful vote, and the mean (0.70) is pulled up by rare reviews with thousands of votes (max 7,663). Review length shows a related pattern: 5-star reviews average just 164 characters, versus 202–283 for 1–4 stars (Table 5) – satisfied customers write less, dissatisfied ones explain themselves at length. Section 5.2 treats this as a feature to derive, not noise to remove.

Table 5: Average review-text length by star rating

Rating	Avg. length (chars)
1 star	202
2 star	275
3 star	280
4 star	283
5 star	164

Table 6 lists the raw schema and pipeline usage. One detail shapes feature engineering (Section 5.2): the 2023 release provides only a raw `helpful_vote` count, not a helpful/total ratio (Amazon no longer surfaces "X of Y found this helpful"), so any helpfulness feature is built from the count itself.

Table 6: Dataset schema

Column	Description
<code>asin / parent_asin</code>	Product ID(s); <code>parent_asin</code> groups product variants
<code>rating</code>	Star rating (1–5)
<code>title / text</code>	Review title and body
<code>helpful_vote</code>	Helpful-vote count
<code>verified_purchase</code>	Verified-purchase flag
<code>timestamp</code>	Submission time (Unix ms)
<code>user_id</code>	Anonymized reviewer ID
<code>Category</code>	Cell Phones / Video Games / Beauty
<code>images</code>	Attached review images, if any

3.4 Scope and Limitations

The extract covers three categories, with Cell Phones representing the large majority (79.7%); Video Games and Beauty results should be read as supporting rather than equally weighted comparisons, and Section 5.3 rankings will report review counts alongside every ranking for this reason. The optional sentiment extension (Objective 7) will use lexicon-based scoring [11] rather than a trained classifier, consistent with course scope.

4 Proposed Methodology

4.1 Data Preprocessing

The extract is largely usable as loaded (missing values affect < 0.04% of records), but four steps precede analysis: (1) convert `timestamp` to Spark `TimestampType` and derive `review_year/review_month`; (2) normalize `verified_purchase` to boolean and range-check `rating` (1–5) via Spark SQL filters; (3) lightly normalize review text (whitespace, lower-casing, markup stripping); (4) drop

duplicate reviews (same `user_id`, `asin`, `timestamp`) and null `text`/`title` rows, logging row counts at each step to catch unexpected data loss.

4.2 Feature Engineering

A compact set of derived features is computed once, early in the pipeline: from `text`, `review_length` and `review_word_count` (motivated by Table 5); from `helpful_vote`, a log-scaled `log_helpful_vote` and a binary `is_highly_helpful` flag; from `timestamp`, `review_year`/`review_month`. At the product level, `product_avg_rating` and `product_review_count` are computed via `groupBy` and joined back for reuse across modules.

4.3 Planned Analytics Modules

Table 7 maps each objective (Section 3) to its module and core PySpark/Spark SQL techniques. Given the manipulation concerns in Section 2, any Objective 2 rating/helpfulness/length difference will be reported descriptively, not as proof of fraud; Objective 7 sentiment scoring is checked against star rating only as an informal agreement rate.

Table 7: Planned analytics modules and core Spark techniques

#	Module	(Objective)	Core Spark techniques
1	Customer Satisfaction		<code>groupBy("rating"), groupBy("Category","rating")</code>
2	Verified-Purchase Analysis		Conditional <code>groupBy("verified_purchase")</code> on <code>rating</code> , <code>helpful_vote</code> , <code>review_length</code>
3	Helpful-Review Analysis		Descending <code>orderBy</code> on <code>helpful_vote</code> ; averages by rating
4	Temporal Analysis	Trend	Spark SQL date functions; time-series aggregation
5	Product Analytics		<code>groupBy</code> on product rating/count with min-count threshold
6	Customer-Activity Analysis		<code>groupBy("user_id")</code> for count and avg. rating
7	Text Analytics		Tokenizer, <code>StopWordsRemover</code> , <code>CountVectorizer</code> / <code>HashingTF</code> + <code>IDF</code> ; optional VADER
8	Window-Function Ranking		<code>Window.partitionBy/orderBy</code> , <code>rank()/dense_rank()</code> , <code>rowsBetween</code>
9	PySpark-vs-Pandas Benchmarking		Timed <code>groupBy</code> , filter, sort, aggregation at multiple sizes

4.4 Performance Benchmarking (Objective 9)

Four core operations – `groupBy` aggregation, sorting, filtering, multi-column aggregation – are implemented in both PySpark and Pandas on identical data, timed with Python’s `time` module across several repetitions to reduce JVM-warm-up/disk-cache noise. Each runs at multiple data sizes (full extract, 10%, 50%) so the benchmark shows a scalability trend, consistent with the Spark literature’s finding that PySpark’s advantage grows with data size [9, 7].

4.5 System Workflow

1. **Ingestion:** 521,607 records loaded into a Spark `DataFrame` with explicit schema (Table 6).

2. **Cleaning/validation:** Spark SQL range/null checks (Section 5.1); row counts logged at each step.
3. **Feature engineering:** Section 5.2 columns computed once and cached.
4. **Analytics modules:** the nine modules in Table 7 run on the cached DataFrame.
5. **Benchmarking:** Section 5.4 comparison runs as a separate track on the same cleaned data.
6. **Results consolidation:** outputs written (Parquet for tables, PNG for plots) and assembled per Section 7.

5 Evaluation Strategy

Most objectives in Section 3 are descriptive rather than predictive, so evaluation centers on correctness and reproducibility rather than accuracy metrics, with the optional sentiment extension evaluated separately.

- **Correctness auditing.** Every aggregation will be spot-checked against Pandas on a sampled subset before the Spark result is trusted at full scale. Row counts are logged before and after every preprocessing step, starting from the confirmed 521,607-record total (Table 1).
- **Benchmark rigor.** The Pandas-versus-PySpark comparison (Section 5.4) will be run multiple times per operation and reported as an average with its spread, with separate cold-start and warm (cached) timings, since Spark’s advantage depends partly on caching [9].
- **Sentiment extension.** If implemented, the VADER pass (Table 7) will be evaluated as a simple agreement rate against star rating, since the extract carries no ground-truth sentiment label.
- **External consistency.** Findings will be checked against the literature in Section 2: the five-star skew against Chevalier and Mayzlin’s positivity finding [3], and any verified/non-verified difference against He, Hollenbeck, and Proserpio’s manipulation concerns [4].

6 Expected Outcomes and Deliverables

The primary technical deliverable is a working, reproducible PySpark pipeline that ingests the 521,607-record extract (Section 4), applies the cleaning and feature-engineering steps (Sections 5.1–5.2), and produces results for all nine objectives in Section 3. Concretely, the project expects to deliver:

- A cleaned, validated dataset with documented row counts at each preprocessing stage.
- A complete picture of customer satisfaction and rating patterns, marketplace-wide and by category (Table 4).
- A quantified comparison of verified- versus non-verified-purchase behaviour across rating, helpfulness, and length.
- A ranked list of the most helpful reviews, with average helpful-vote count overall and by rating.
- Year-over-year insight into review volume and rating trends across 1999–2023, including the rise around 2019–2021.
- Rankings of the highest- and lowest-rated products, the most-reviewed products, and the most active reviewers.

- Text-based findings, including common keywords, the length-by-rating pattern (Table 5), and, time permitting, a lexicon-based sentiment pass.
- A working demonstration of Spark window functions for product ranking, reviewer ranking, and rolling-average rating.
- An empirical, quantified performance-comparison table showing PySpark’s runtime relative to Pandas at more than one data size.

Taken together, these deliverables give a product or marketing team something actionable rather than a pile of intermediate statistics, while giving this course’s evaluators a complete demonstration of the PySpark concepts in Table 7.

7 Conclusion

This report has presented a proposal, grounded in confirmed dataset statistics rather than assumption alone, for a large-scale customer-review analytics system built on Apache PySpark. Working with a real 521,607-record extract of the Amazon Reviews 2023 dataset spanning three categories and 24 years, this project has confirmed the extract’s schema, its category composition (Cell Phones dominant at 79.7%), its rating distribution (a five-star skew of 61.3%, consistent with prior findings on the positivity of online reviews), its verified-purchase split (93.1% verified), and a less obvious pattern worth carrying into the pipeline: five-star reviews run noticeably shorter than lower-rated ones. Building on this foundation, the project proposes a PySpark pipeline covering customer satisfaction, verified-purchase behaviour, helpful-review patterns, temporal trends, product- and reviewer-level analytics, and text analytics, implemented with Spark SQL and Spark’s window-function API, and closing with an empirical, timed comparison against Pandas. Every objective in Section 3 is directly supported by a field already confirmed present in the dataset (Table 6), giving this project’s central technical argument, that PySpark is the appropriate tool for a dataset at this scale, concrete and falsifiable evidence rather than assertion, fulfilling the learning objectives of the Data Analytics Lab course.

References

- [1] Hou, Y., Li, J., He, Z., Yan, A., Chen, X., and McAuley, J. Bridging Language and Items for Retrieval and Recommendation. *arXiv preprint arXiv:2403.03952*, 2024.
- [2] McAuley Lab. Amazon Reviews 2023 Dataset. Hugging Face Datasets, 2023. Available at: <https://huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023>
- [3] Chevalier, J. A., and Mayzlin, D. The Effect of Word of Mouth on Sales: Online Book Reviews. *Journal of Marketing Research*, 43(3), 345–354, 2006.
- [4] He, S., Hollenbeck, B., and Proserpio, D. The Market for Fake Reviews. *Marketing Science*, 41(5), 896–921, 2022.
- [5] Saumya, S., Roy, P. K., and Singh, J. P. Review Helpfulness Prediction on E-commerce Websites: A Comprehensive Survey. *Engineering Applications of Artificial Intelligence*, 126, 107075, 2023.
- [6] Khatun, M. E., Akter, H., Rehan, T., and Ahmed, T. Leveraging Big Data Frameworks for Spam Detection in Amazon Reviews. *arXiv preprint arXiv:2509.21579*, 2025.
- [7] Gupta, Y. K., and Kumari, S. A Study of Big Data Analytics Using Apache Spark with Python and Scala. In *2020 3rd International Conference on Intelligent Sustainable Systems (ICISS)*, IEEE, 471–478, 2020.